SOFTWARE ENGINEERING · CHAPTER 08

Lab-08: Class Diagrams

You continue the ITC Campus Library System from Lab-07. From its use cases you build a domain model, add associations with multiplicities and navigability, introduce generalization, interfaces and an abstract fee policy, organise everything into packages and implement the domain package in Java 25 with JPA annotations that match the diagram line for line. Task 5 turns a real "god class" into a SOLID design. The challenge reverse-engineers a class diagram from an existing Spring codebase, reconciles it with your design and documents two decisions as ADRs.

UML 2.5.1 · Mermaid 11 JDK 25 · Maven 3.9 · Spring Boot 4.1.1 · Hibernate 7.4 ≈ 5 hours 5 tasks + 1 challenge

How to work through this lab. Do the tasks in order; each builds on the previous. Read the *Goal*, follow the *Steps*, compare with the *Expected output*, and tick the *Acceptance checklist* (saved in your browser). Open a hint only when stuck for more than 10 minutes. Every diagram is submitted twice: as **source text** (Mermaid `.mmd` or PlantUML `.puml`) and as a **PNG/SVG export**. You may draw in draw.io instead, but then you also write the Mermaid source by hand.

0 Setup — 20 min

1. Verify the tooling. Mermaid CLI needs Node 20+; PlantUML needs Java only.

```
java -version        # openjdk 25
mvn -v               # Apache Maven 3.9.16
node -v              # v20 or newer (for the Mermaid CLI)
npx -y @mermaid-js/mermaid-cli -V     # 11.x, downloads on first run
```

2. Create the lab folder. Diagrams live next to the code, as text.

```
lab-08/
├── README.md                noun table, justification tables, SOLID table,
ADRs
├── diagrams/
│   ├── task1-domain.mmd      + task1-domain.png
│   ├── task2-associations.mmd
│   ├── task3-hierarchy.mmd
```

```
    │   ├── task4-packages.mmd
    │   ├── task5-before.mmd
    │   ├── task5-after.mmd
    │   └── challenge-design.mmd
    └── library/                  Spring Boot 4.1.1 project (Tasks 3-5,
challenge)
```

3. Generate `library/` at start.spring.io: Maven, Java 25, Spring Boot 4.1.1, group `edu.itc`, artifact `library`, dependencies *Spring Data JPA*, *Spring Web MVC*, *Validation*, *H2 Database*. The relevant part of the `pom.xml`:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
</parent>
<properties><java.version>25</java.version></properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId><artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

4. Package layout inside `library/src/main/java/edu/itc/library/`: `web`, `service`, `domain`, `repository`, `infrastructure`. Create the folders now; Task 4 fills them.
5. Render a diagram (repeat for every `.mmd`; the command is also in the Submission section):

```
cd lab-08
npx -y @mermaid-js/mermaid-cli -i diagrams/task1-domain.mmd \
```

```
          -o diagrams/task1-domain.png -b transparent -s 2
```

6. Recap of the Lab-07 use cases you model in this lab. Keep this table open; every class you draw must be traceable to at least one row.

Use case	Primary actor	Main success scenario (short)
UC-01 Register member	Librarian	Creates a member record as *student* (student number, programme) or *staff* (department); the system e-mails a welcome message.
UC-02 Search catalogue	Member	Searches books by title, author or ISBN; sees how many copies are available and on which shelf.
UC-03 Borrow copy	Member, Librarian	Librarian scans the copy barcode and member card; the system checks the member's loan limit (students 5, staff 10) and unpaid fines, creates a loan with a due date (students 14 days, staff 30 days) and marks the copy on loan.
UC-04 Return copy	Librarian	Closes the loan; if overdue, a fine is issued according to the fee policy; if the book is reserved, the first reservation is notified.
UC-05 Reserve book	Member	Places a reservation on a book with no available copy; reservations are queued in order.
UC-06 Pay fine	Member	Pays an open fine by card through the payment gateway; the fine is marked paid.
UC-07 Send overdue notice	System (scheduled)	Every night, members with overdue loans receive a notice by e-mail or SMS according to their preference.
UC-08 Manage catalogue	Librarian	Adds books and copies, changes shelf locations, marks copies lost or withdrawn.

1 Domain model from the use cases

Easy — 40 min

GOAL

Turn the nouns of the eight use cases into an analysis-level class diagram: 8 to 12 classes with typed attributes, value objects and enumerations marked, no associations yet and no getters or setters.

STEPS

1. Read the use-case table and underline every noun. Write a **noun table** in `README.md` with three columns: noun, decision (*class* / *attribute of …* / *discard*), reason. At least 15 nouns (for example *book, copy, barcode, member, student, staff, loan, due date, reservation, fine, fee policy, librarian, catalogue, notice, payment gateway, card*).
2. Create `diagrams/task1-domain.mmd` starting from the starter below. One `class` block per class: expect `Book`, `Copy`, `Member`, `Loan`, `Reservation`, `Fine`, `Librarian`, `Money`, plus the enumerations `CopyStatus` and `ReservationStatus`. Student and staff appear in Task 3.
3. Give every attribute a type and the `-` visibility: `-isbn: Isbn`, `-title: String`, `-publishedYear: int`, `-barcode: String`, `-shelf: String`, `-borrowedOn: LocalDate`, `-dueOn: LocalDate`, `-returnedOn: LocalDate`, `-amount: Money`. Optional attributes get a note or a trailing `[0..1]` in the README.
4. Mark value objects with `<<record>>` (`Money`, `Isbn`, `Email`) and enumerations with `<<enumeration>>` listing their literals (`AVAILABLE, ON_LOAN, RESERVED, LOST, WITHDRAWN`).
5. Add at most two operations per class, and only where a use case demands behaviour: `Loan.isOverdue(today: LocalDate) boolean`, `Copy.markLost() void`, `Fine.markPaid() void`. No `getX()`/`setX()`.
6. Render `task1-domain.png` and check that every class in the diagram appears in the noun table with decision *class*.
7. Write three sentences in the README on what you discarded and why (for example: *Catalogue* is the set of all books, not a class; *Notice* is a message that is sent, not stored).

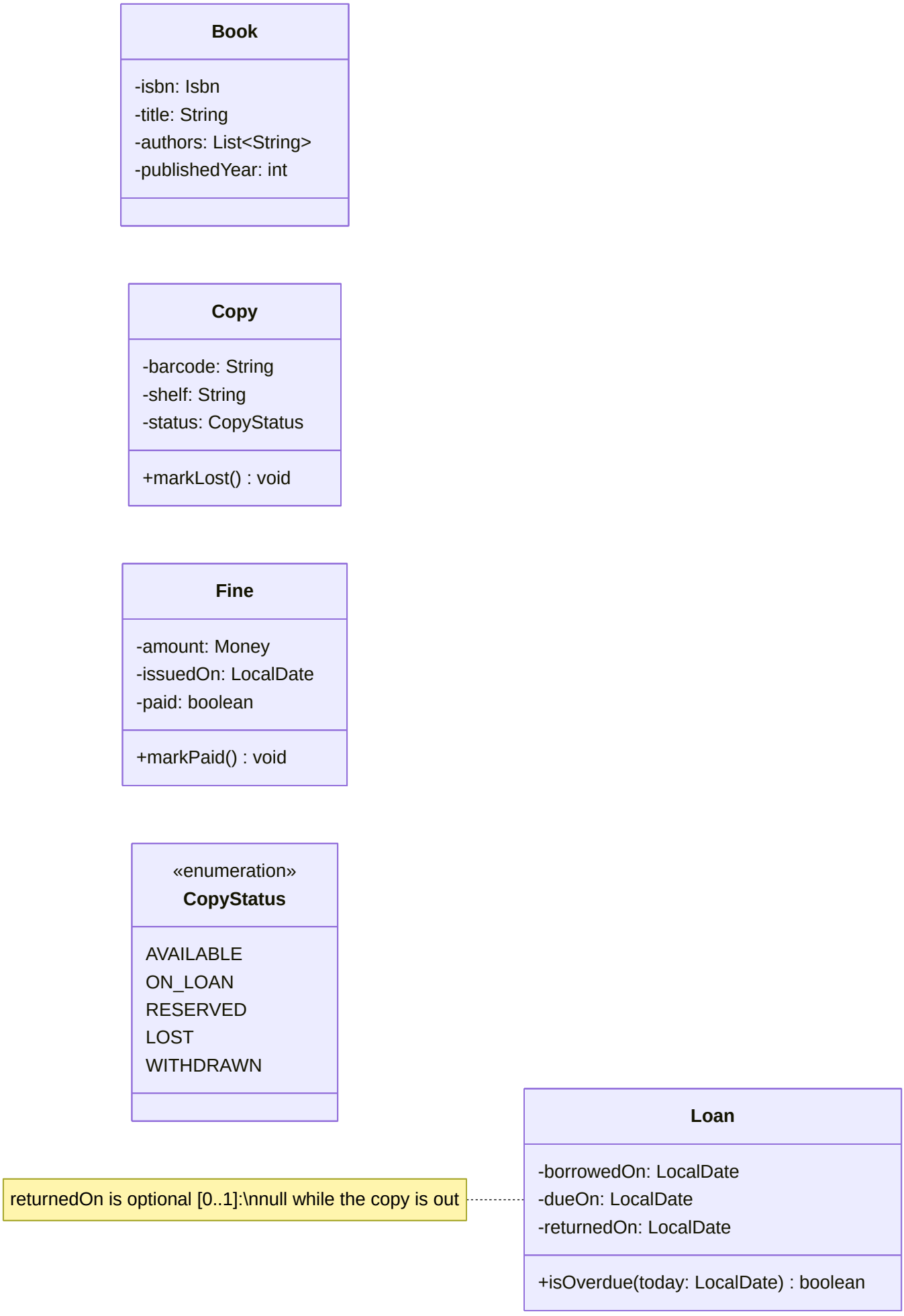
STARTER DIAGRAM TEXT

```
classDiagram
  direction LR
  class Book {
    -isbn: Isbn
    -title: String
    -authors: List~String~
    -publishedYear: int
  }
  class Copy {
    -barcode: String
    -shelf: String
    -status: CopyStatus
    +markLost() void
  }
  class CopyStatus {
    <<enumeration>>
    AVAILABLE
    ON_LOAN
```

```
    RESERVED
    LOST
    WITHDRAWN
}
class Money {
    <<record>>
    -amount: BigDecimal
    -currency: String
}
%% TODO: Member, Loan, Reservation, Fine, Librarian,
%% TODO: ReservationStatus, Isbn, Email
```

EXPECTED OUTPUT

Your diagram has 8 to 12 boxes and no lines. The part below shows the expected level of detail for three of them (your names may differ slightly, your types must not be missing).


Book
-isbn: Isbn
-title: String
-authors: List<String>
-publishedYear: int
Copy
-barcode: String
-shelf: String
-status: CopyStatus
+markLost() : void
Fine
-amount: Money
-issuedOn: LocalDate
-paid: boolean
+markPaid() : void
«enumeration»
CopyStatus
AVAILABLE
ON_LOAN
RESERVED
LOST
WITHDRAWN
Loan
-borrowedOn: LocalDate
-dueOn: LocalDate
-returnedOn: LocalDate
+isOverdue(today: LocalDate) : boolean
returnedOn is optional [0..1]:\nnull while the copy is out

Show hints

- Class or attribute? If the noun has attributes of its own or a lifecycle (created, changed, closed), it is a class. *Due date* has neither: it is `Loan.dueOn`. *Copy* has a barcode, a shelf and a status: it is a class.
- `Book` is the bibliographic record (one per ISBN); `Copy` is the physical item (one per barcode). UC-02 "how many copies are available" only works if both exist.
- Use `LocalDate` for calendar dates (due date), `Instant` for points in time (when a reservation was placed).
- Mermaid comments start with `%%`. Generic types use `List~String~`, not angle brackets.
- *Librarian* is an actor in Lab-07 but also data here: UC-03 records who issued the loan. Keep it as a class with `-badgeNo: String`.

ACCEPTANCE CHECKLIST

- [ ] 8 to 12 classes, every attribute typed with `-` visibility; `Book` and `Copy` are separate classes.
- [ ] Value objects carry `<<record>>`, enumerations carry `<<enumeration>>` with their literals.
- [ ] No getter or setter appears; every operation is demanded by a use case.
- [ ] `task1-domain.mmd`, `task1-domain.png` and the noun table (15+ rows) are committed.

2 Associations, multiplicity and navigability **Medium** **45 min**

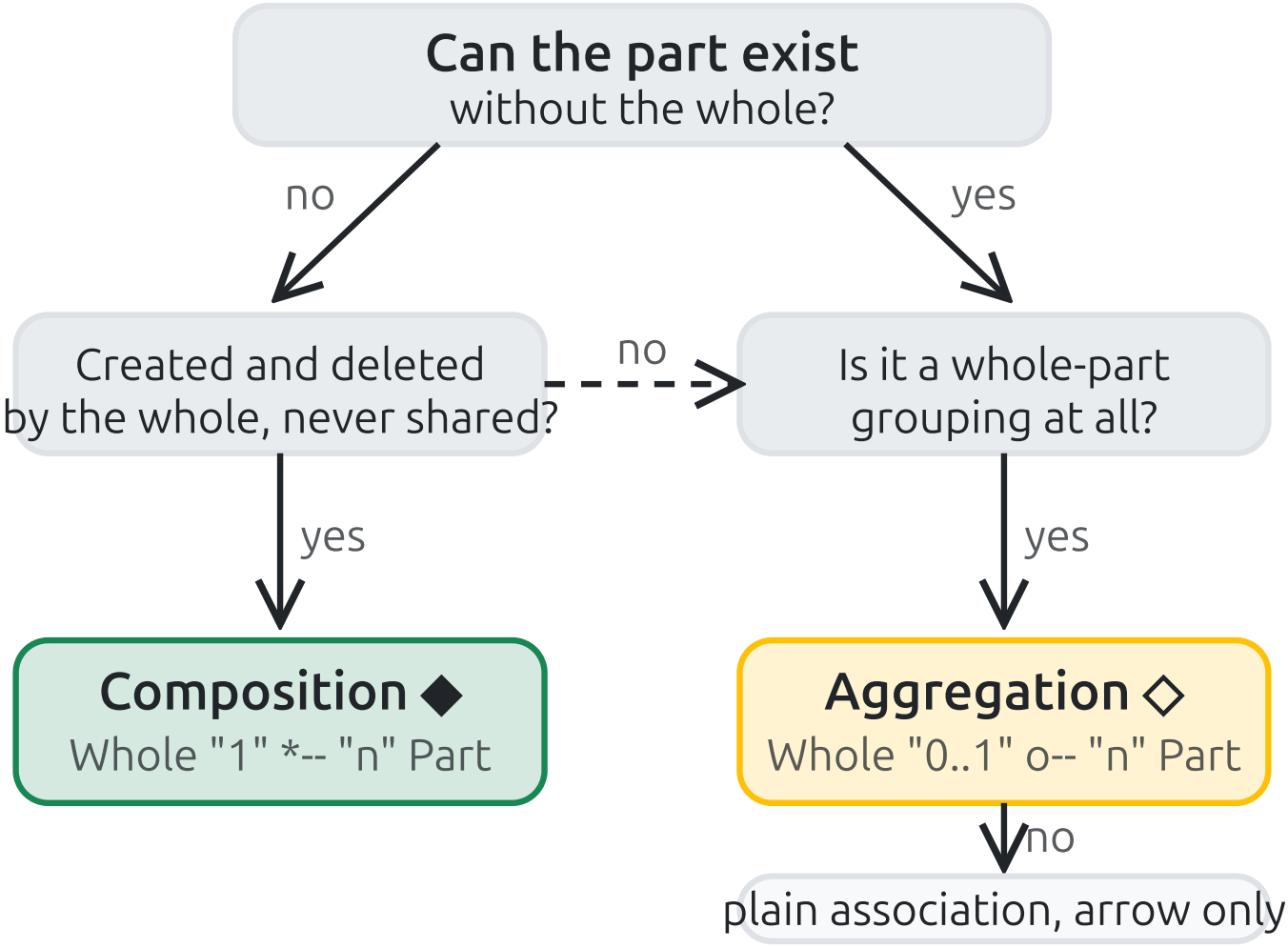
GOAL

Connect the classes: every association gets a role label, a multiplicity at *both* ends and a navigability decision. Decide, and justify in a table, whether Book/Copy and Member/Loan are compositions, aggregations or plain associations.

STEPS

1. Copy `task1-domain.mmd` to `diagrams/task2-associations.mmd`. Add one relation line for each pair that the use cases link: Book/Copy, Member/Loan, Loan/Copy, Loan/Librarian (*issuedBy*), Reservation/Member, Reservation/Book, Fine/Loan, Fine/Member. Give every line a multiplicity at both ends in quotes and a role label after the colon.
2. Decide **navigability** from the use cases. UC-03 needs the member's number of active loans, so `Member` must reach its loans: `Member "1" --> "0..*" Loan : activeLoans`. UC-04 needs the loan's copy and member: arrows from `Loan`. Where nothing needs the reverse direction, draw a one-way arrow, not a plain line.

3. Decide the **diamond** for Book/Copy and Member/Loan. Fill in the justification table in the README (skeleton below) by answering the four questions from the slides for each pair. Use `*--`, `o--` or a plain arrow accordingly.
4. Reservations are queued: label the Book/Reservation end `{ordered}` in a note and give it multiplicity `0..*`. A member may hold at most three reservations: `"0..3"`.
5. Check for the "same field twice" mistake: an attribute `-copy: Copy` inside `Loan` *and* a line `Loan --> Copy` is wrong. Remove the attribute; keep value-object attributes (`Money`, `Isbn`) in the compartments.
6. Add a reflexive or a second association if your noun table demands it (for example `Book --> Author` if you made Author a class). Every line must be traceable to a use case.
7. Render `task2-associations.png`. Count: lines without a multiplicity at both ends must be zero.



STARTER: RELATION LINES AND JUSTIFICATION TABLE

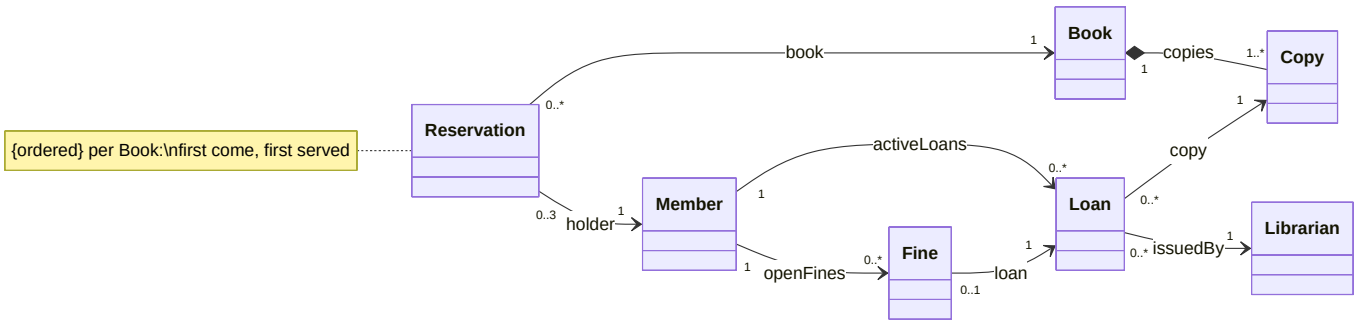
```
%% replace every ? ; the left operand of *-- / o-- is the whole
Book "1" ?-- "1..*" Copy : copies
Member "1" --> "0..*" Loan : activeLoans
```

```
    Loan "0..*" --> "1" Copy : copy
    Loan "0..*" --> "1" Librarian : issuedBy
    Reservation "0..*" --> "1" Book : book
    Reservation "0..3" --> "1" Member : holder
    Fine "0..1" --> "1" Loan : loan
    ? "?" ?-- "?" ? : ?

  README.md
  | Pair          | Part exists alone? | Shared? | Who creates? | Delete cascades?
  | Decision |
  |---------------|--------------------|---------|--------------|-----------------
  -|----------|
  | Book / Copy   |                    |         |              |
  |          |
  | Member / Loan |                    |         |              |
  |          |
```

EXPECTED OUTPUT

One defensible solution (attributes hidden here to keep the picture small; yours keeps them). Other diamonds are acceptable if your table justifies them.



Show hints

- Book/Copy: can a copy be moved to a different book record? No, a copy *is* a physical instance of that book. Withdrawing the book withdraws all copies. That is the textbook composition.
- Member/Loan: a loan without a member is meaningless, but a library keeps loan history after a member leaves (and privacy rules may demand anonymising, not deleting). Most teams choose a plain navigable association, not composition. Argue it in the table; either answer with a good argument earns the points.
- Mermaid: `Member "1" --> "0..*" Loan : activeLoans` puts `1` at Member, `0..*` at Loan and the arrowhead at Loan. The label is the role at the arrow end.
- Navigability is about the code you will write in Task 4: an arrow from A to B means class A gets a field of type B (or a collection of B).
- `Fine "0..1" --> "1" Loan` reads "a fine belongs to exactly one loan; a loan has at most one fine".

ACCEPTANCE CHECKLIST

- [ ] Every association has a multiplicity at both ends and a role label; no bare lines.
- [ ] Navigability follows the use cases (Member reaches its active loans; Loan reaches Copy and Member) and no field is shown twice as attribute and line.
- [ ] The justification table answers all four questions for Book/Copy and Member/Loan, and the diamonds in the diagram match the table.
- [ ] `task2-associations.mmd` and `.png` are committed and render without errors.

3 Generalization, interfaces and abstract classes **Medium** **50 min**

GOAL

Model the variation in the domain: `Member` becomes abstract with `Student` and `Staff` subclasses that answer the loan limit polymorphically, `NotificationChannel` becomes an interface with two realizations, and the fine calculation becomes an abstract `FeePolicy` with a template method. Then write the Java that matches, without JPA yet.

STEPS

1. Copy Task 2 to `diagrams/task3-hierarchy.mmd`. Make `Member` abstract (`<<abstract>>`) with two abstract operations `+maxLoans()* int` and `+loanPeriodDays()* int`, and one concrete `+canBorrow() boolean` that compares `activeLoans` with `maxLoans()`. Add `Student` (`-studentNo: String`, `-programme: String`; 5 loans, 14 days) and `Staff` (`-department: String`; 10 loans, 30 days) with `Member <|-- Student` and `Member <|-- Staff`.
2. Add `<<interface>> NotificationChannel` with `+send(to: Member, n: Notification) void` and `+supports(pref: ContactPreference) boolean`. Realize it with `EmailChannel` and `SmsChannel` (`NotificationChannel <|.. EmailChannel`). A `NotificationService` holds `"1..*"` channels.
3. Add `<<abstract>> FeePolicy` with the template method `+compute(loan: Loan, today: LocalDate) Money` and the abstract steps `#fee(daysLate: long)* Money` and `#cap()* Money`. Subclasses: `DailyFeePolicy` (0.50 per day, cap 20.00) and `FlatFeePolicy` (5.00 once, no cap). `FineService --> FeePolicy`.
4. Add `<<interface>> PaymentGateway` with `+charge(amount: Money, token: String) Receipt`, realized by `StripePaymentGateway`; `FineService --> PaymentGateway`. Check every triangle: it sits at the parent or at the interface.
5. Implement the hierarchy in `library/src/main/java/edu/itc/library/domain/` from the starter: `Member`, `Student`, `Staff`, `FeePolicy`, `DailyFeePolicy`, `FlatFeePolicy`, `Money`,

`NotificationChannel`, `PaymentGateway`. Plain Java, no annotations. `mvn -q compile` must pass.

6. Write `FeePolicyTest` (JUnit 6): a loan 3 days late costs `1.50` with the daily policy and `5.00` with the flat policy; a loan 100 days late is capped at `20.00`; a loan returned on time costs `0.00` with both.
7. In the README, one paragraph: why `Member` is an abstract class and not an interface, and why `NotificationChannel` is an interface and not an abstract class. Render `task3-hierarchy.png`.

STARTER CODE

```
package edu.itc.library.domain;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public abstract class Member {
    private final String name;
    private final Email email;
    private final List<Loan> activeLoans = new ArrayList<>();

    protected Member(String name, Email email) {
        this.name = name;
        this.email = email;
    }

    public abstract int maxLoans();          // Student 5, Staff 10
    public abstract int loanPeriodDays();    // Student 14, Staff 30

    public boolean canBorrow() {
        return activeLoans.size() < maxLoans();   // TODO: also no open fines
    }
    public List<Loan> activeLoans() { return List.copyOf(activeLoans); }
}

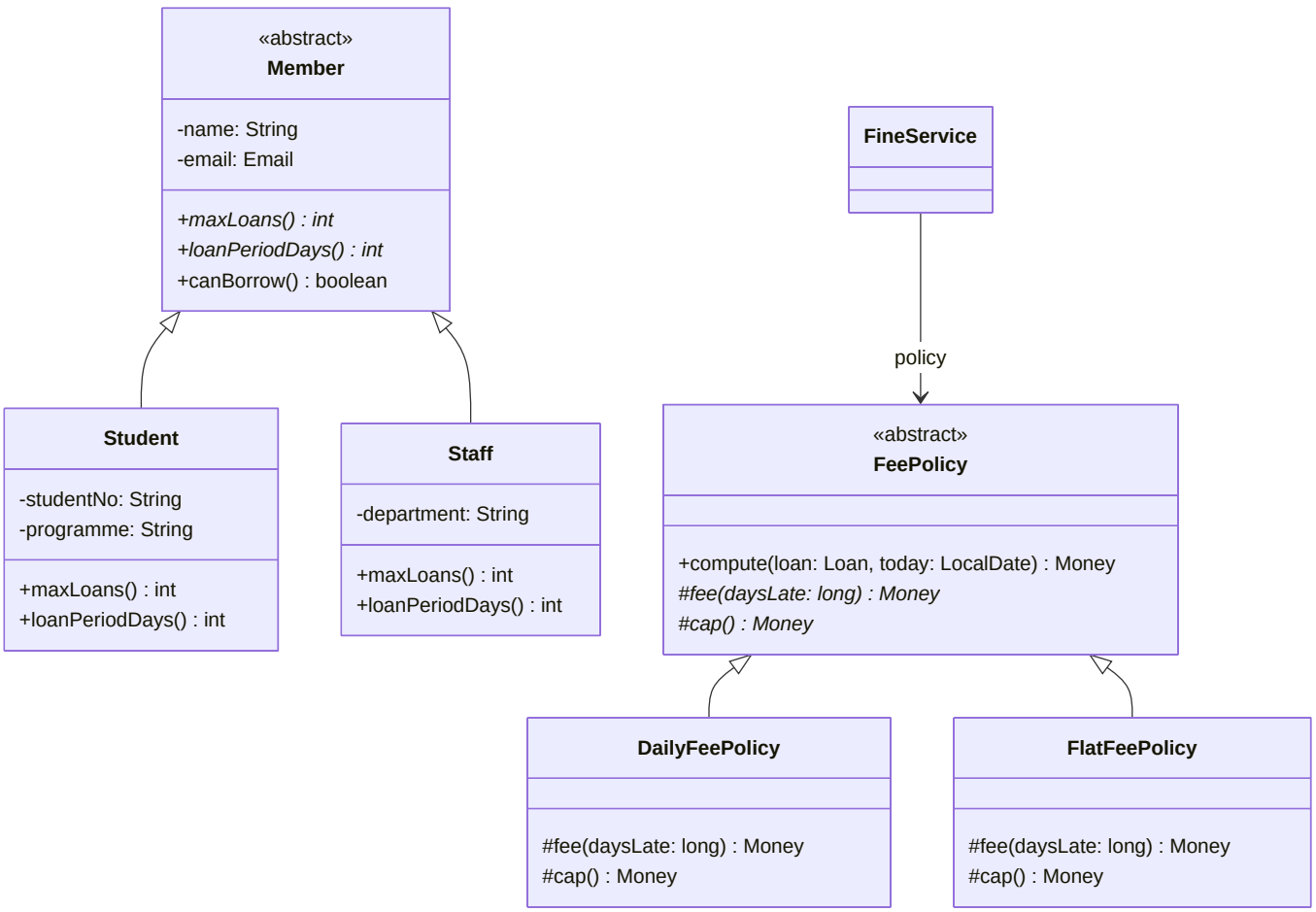
public abstract class FeePolicy {
    // template method: fixed steps, variable amounts
    public final Money compute(Loan loan, LocalDate today) {
        long daysLate = loan.daysLate(today);
        if (daysLate <= 0) return Money.ZERO;
        return Money.min(fee(daysLate), cap());
    }
    protected abstract Money fee(long daysLate);
    protected abstract Money cap();
}
```

```
public record Money(BigDecimal amount, String currency) {
    public static final Money ZERO = Money.of("0.00");
    public static Money of(String amount) {
        return new Money(new BigDecimal(amount), "EUR");
    }
    public Money times(long factor) {
        return new Money(amount.multiply(BigDecimal.valueOf(factor)), currency);
    }
    public static Money min(Money a, Money b) {
        return a.amount.compareTo(b.amount) <= 0 ? a : b;
    }
    // TODO: add(Money), isPositive()
}

public interface NotificationChannel {
    boolean supports(ContactPreference preference);
    void send(Member to, Notification notification);
}
```

EXPECTED OUTPUT

The hierarchy part of your diagram should read like this (channels and gateway omitted here):



```
$ mvn -q test -Dtest=FeePolicyTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
```

Show hints

- Mermaid marks an abstract operation with `*` after the parentheses: `+maxLoans()* int`. The concrete override in `Student` has no star.
- `Loan.daysLate(today)` is `ChronoUnit.DAYS.between(dueOn, today)`, negative or zero when on time. Put it in `Loan`: the loan knows its own dates (responsibility first).
- `FlatFeePolicy.cap()` can return a very large amount or the fee itself; document the choice. Do not return `null`.
- Abstract class vs interface: `Member` has state (name, active loans) and a shared algorithm (`canBorrow()`); a channel has no state and a class might support several contracts. That is the paragraph.
- `FineService` and `NotificationService` are design-level classes; it is fine to introduce them now, they move to the `service` package in Task 4.

ACCEPTANCE CHECKLIST

- [ ] `Member` is abstract with two abstract operations; `Student` and `Staff` override them with 5/14 and 10/30; triangles point at `Member`.
- [ ] `NotificationChannel` and `PaymentGateway` are `<<interface>>` with dashed realizations from their implementations.
- [ ] `FeePolicy.compute()` is a final template method; `FeePolicyTest` passes with 4 tests.
- [ ] Java classes match the diagram name for name and signature for signature; the README paragraph is written.

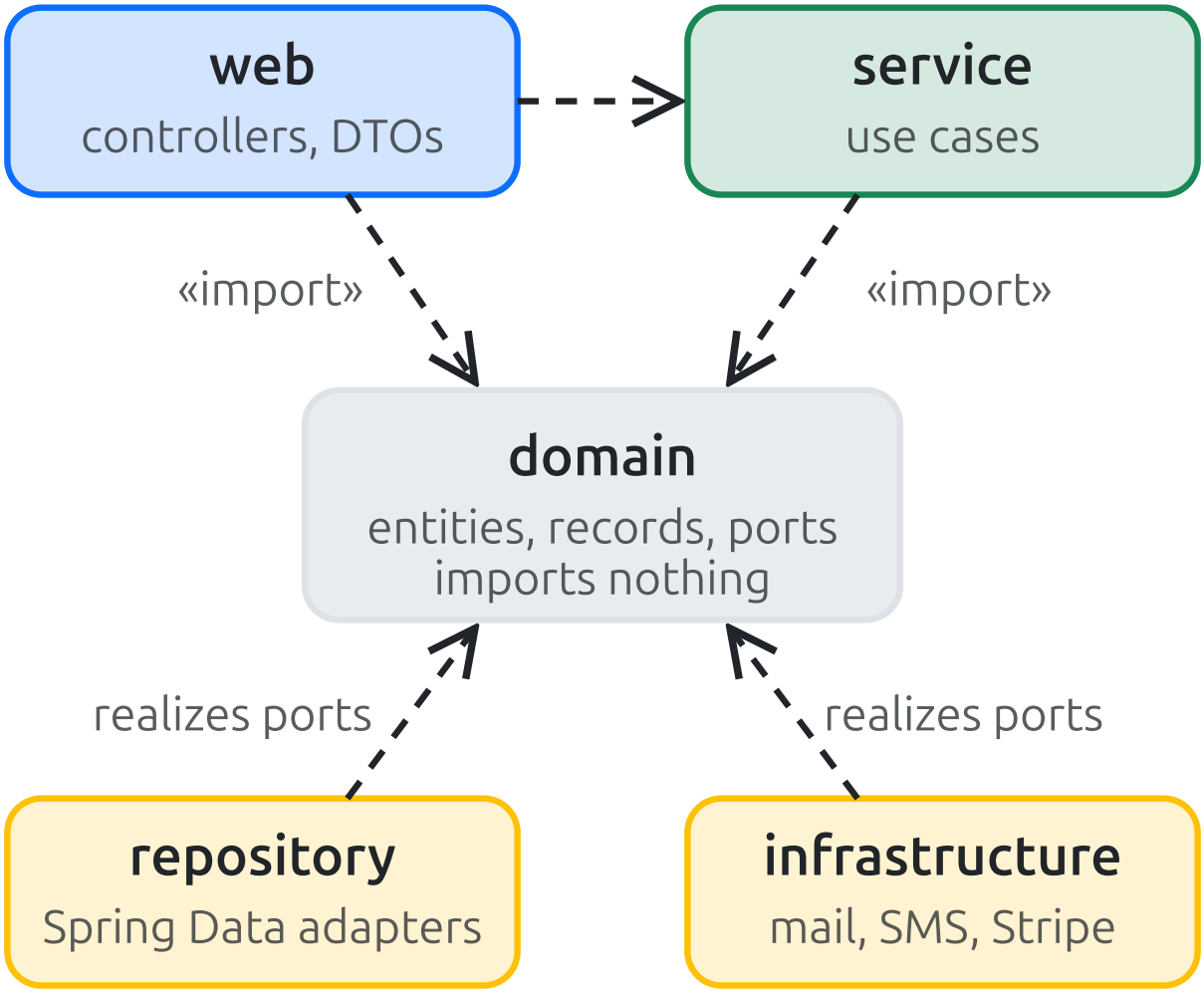
4 Packages and implementation mapping Hard 75 min

GOAL

Organise the design into packages with inward-pointing dependencies, then implement the `domain` package in Java 25 with records, a sealed result type and JPA annotations that translate the diagram exactly (composition, inheritance, value objects). A unit test proves the loan-limit rule without Spring.

STEPS

1. Create `diagrams/task4-packages.mmd` with five `namespace` blocks: `web` (`LoanController`, `MemberController`, DTO records), `service` (`LoanService`, `FineService`, `NotificationService`), `domain` (all Task 3 classes plus the interfaces `LoanRepository`, `MemberRepository`, `CopyRepository`, `NotificationChannel`, `PaymentGateway`, `FeePolicy`), `repository` (`JpaLoanRepository`, …) and `infrastructure` (`EmailChannel`, `SmsChannel`, `StripePaymentGateway`). Draw the relations between packages' classes. Rule: no arrow leaves `domain`; `service` never points at `repository` or `infrastructure`.
2. Implement `domain` with JPA: `Member` is `@Entity @Inheritance(strategy = InheritanceType.JOINED)` and abstract; `Student`/`Staff` are `@Entity`; `Money` and `Email` are `@Embeddable` records; `Book "1" *-- "1..*" Copy` becomes `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true) @JoinColumn(name = "book_isbn")`; `Loan` has `@ManyToOne(optional = false)` to `Copy` and `Member`.
3. Add `sealed interface BorrowResult permits Borrowed, Rejected` with two records. `Member.borrow(Copy copy, LocalDate today)` returns `Rejected("loan limit 5 reached")` or `Rejected("copy not available")`, otherwise creates the `Loan` with `dueOn = today.plusDays(loanPeriodDays())`, marks the copy `ON_LOAN`, adds the loan to `activeLoans` and returns `Borrowed(loan)`.
4. Repositories: interfaces in `domain` (`LoanRepository { Loan save(Loan l); List<Loan> findOverdue(LocalDate today); }`); Spring Data adapters in `repository`: `interface JpaLoanRepository extends JpaRepository<Loan, UUID>, LoanRepository` with a `@Query` for `findOverdue`.
5. Write `MemberLoanLimitTest` (plain JUnit 6, no Spring context): a `Student` that already holds 5 loans gets `Rejected`, a `Staff` with 5 loans gets `Borrowed`, a student's `dueOn` is `today + 14`, and a copy that is `ON_LOAN` is rejected for anyone. Use a `switch` with pattern matching over `BorrowResult` in the assertions.
6. Run `mvn -q test`, then `mvn -q spring-boot:run` with `spring.jpa.show-sql=true` and H2: the log shows the `JOINED` tables `member`, `student`, `staff` and the `book_isbn` foreign key on `copy`.
7. In the README, a translation table with at least eight rows: diagram element → Java construct → JPA annotation (as on slide 21), plus every deviation JPA forced (non-final entity, protected no-arg constructor).



STARTER CODE

```
package edu.itc.library.domain;

import jakarta.persistence.*;
import java.time.LocalDate;
import java.util.*;

public sealed interface BorrowResult permits Borrowed, Rejected {}
public record Borrowed(Loan loan) implements BorrowResult {}
public record Rejected(String reason) implements BorrowResult {}

@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Member {
    @Id private UUID id;
    private String name;
    @Embedded private Email email;

    // Member "1" --> "0..*" Loan : activeLoans  (navigable from Member)
    @OneToMany(mappedBy = "member")
```

```
    private List<Loan> loans = new ArrayList<>();

    protected Member() {}                       // JPA only
    protected Member(UUID id, String name, Email email) {
        this.id = id; this.name = name; this.email = email;
    }

    public abstract int maxLoans();
    public abstract int loanPeriodDays();

    public List<Loan> activeLoans() {
        return loans.stream().filter(l -> l.returnedOn() == null).toList();
    }

    public BorrowResult borrow(Copy copy, LocalDate today) {
        // TODO 1: activeLoans().size() >= maxLoans()  -> Rejected
        // TODO 2: copy.status() != CopyStatus.AVAILABLE -> Rejected
        // TODO 3: new Loan(UUID.randomUUID(), this, copy, today,
        //                  today.plusDays(loanPeriodDays()))
        //         copy.markOnLoan(); loans.add(loan); return new Borrowed(loan)
        throw new UnsupportedOperationException("TODO");
    }
}

@Entity
public class Student extends Member {
    private String studentNo;
    private String programme;
    protected Student() {}
    public Student(UUID id, String name, Email email, String studentNo) {
        super(id, name, email); this.studentNo = studentNo;
    }
    @Override public int maxLoans() { return 5; }
    @Override public int loanPeriodDays() { return 14; }
}

@Entity
public class Book {
    @Id private String isbn;
    private String title;
    // Book "1" *-- "1..*" Copy : copies   (composition)
    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "book_isbn", nullable = false)
    private List<Copy> copies = new ArrayList<>();

    public Copy addCopy(String barcode, String shelf) {
        Copy c = new Copy(barcode, shelf);      // the whole creates the part
        copies.add(c);
```

```
            return c;
        }
        public List<Copy> copies() { return List.copyOf(copies); }
    }

    // src/test/java/edu/itc/library/domain/MemberLoanLimitTest.java
    class MemberLoanLimitTest {
        LocalDate today = LocalDate.of(2026, 9, 25);

        @Test void studentCannotExceedFiveLoans() {
            Member ann = new Student(UUID.randomUUID(), "Ann",
                                     new Email("ann@itc.edu"), "S-1001");
            Book book = new Book("978-0134685991", "Effective Java");
            for (int i = 0; i < 5; i++) {
                assertInstanceOf(Borrowed.class,
                    ann.borrow(book.addCopy("BC-" + i, "A1"), today));
            }
            BorrowResult sixth = ann.borrow(book.addCopy("BC-6", "A1"), today);
            switch (sixth) {
                case Rejected r -> assertEquals("loan limit 5 reached", r.reason());
                case Borrowed b -> fail("sixth loan must be rejected");
            }
        }
        // TODO: staffMayHoldTenLoans, studentDueDateIsFourteenDays,
        //       copyOnLoanIsRejected
    }
```

EXPECTED OUTPUT

```
$ mvn -q test
[INFO] Running edu.itc.library.domain.MemberLoanLimitTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running edu.itc.library.domain.FeePolicyTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0

$ mvn -q spring-boot:run          (excerpt, spring.jpa.show-sql=true)
Hibernate: create table member (id uuid not null, name varchar(255),
           email varchar(255), primary key (id))
Hibernate: create table student (student_no varchar(255), programme
varchar(255),
           id uuid not null, primary key (id))
Hibernate: create table staff (department varchar(255), id uuid not null,
           primary key (id))
Hibernate: create table copy (barcode varchar(255) not null, shelf varchar(255),
           status varchar(255) check (status in ('AVAILABLE','ON_LOAN',
           'RESERVED','LOST','WITHDRAWN')), book_isbn varchar(255) not null,
           primary key (barcode))
```

```
Hibernate: alter table if exists copy add constraint FK... foreign key
           (book_isbn) references book
Hibernate: alter table if exists student add constraint FK... foreign key (id)
           references member
```

Show hints

- Records as `@Embeddable` are supported by Hibernate 7 (Jakarta Persistence 3.2). Use field names that map to sensible columns: `Email(String address)` gives column `address`; add `@AttributeOverride` if you prefer `email`.
- With `JOINED`, only the base class carries `@Id`; subclasses get a table with the same primary key. Keep the `protected` no-arg constructors: JPA needs them, the diagram does not show them.
- In the unit test nothing is persisted; the domain classes work as plain objects because the annotations are inert without a persistence context. That is the point of keeping rules in the domain.
- A sealed interface plus records lets the test `switch` without a `default` branch; if you add a third result type the test stops compiling, which is what you want.
- Spring Data derives `JpaLoanRepository`; the domain interface it extends is what `LoanService` injects. If `LoanService` imports anything from `edu.itc.library.repository`, you have a DIP violation and an arrow in the wrong direction.

ACCEPTANCE CHECKLIST

- [ ] `task4-packages.mmd` uses five namespaces; no relation leaves `domain`, and `service` depends only on `domain`.
- [ ] JPA mapping translates the diagram: `JOINED` inheritance, composition with `cascade = ALL, orphanRemoval = true`, `@Embeddable` records, `@ManyToOne(optional = false)` for multiplicity 1.
- [ ] `MemberLoanLimitTest` (4 tests, no Spring) and `FeePolicyTest` pass with `mvn -q test`; `BorrowResult` is sealed.
- [ ] The application starts on H2 and the README translation table has 8+ rows plus the list of JPA-forced deviations.

5 SOLID review of `LibraryManager` Hard 60 min

GOAL

A previous team shipped the library as one class. Draw it as it is, name every SOLID violation, design the replacement as a class diagram, and map each change to the principle that

motivated it.

THE GOD CLASS (GIVEN)

```
package edu.itc.library.legacy;

@Service
public class LibraryManager {
    private final JdbcTemplate jdbc;              // SQL for every table
    private final JavaMailSender mail;            // e-mails
    private final TwilioClient sms;               // SMS
    private final StripeClient stripe;            // card payments
    private final PdfWriter pdf;                  // monthly report

    public void borrow(String memberId, String barcode) {
        Map<String, Object> m = jdbc.queryForMap(
            "select * from member where id = ?", memberId);
        int limit = "STUDENT".equals(m.get("type")) ? 5
                  : "STAFF".equals(m.get("type")) ? 10 : 0;
        int active = jdbc.queryForObject(
            "select count(*) from loan where member_id = ? "
            + "and returned_on is null", Integer.class, memberId);
        if (active >= limit) throw new IllegalStateException("limit");
        int days = "STUDENT".equals(m.get("type")) ? 14 : 30;
        jdbc.update("insert into loan(...) values (...)", /* ... */ days);
        jdbc.update("update copy set status = 'ON_LOAN' where barcode = ?",
                    barcode);
        mail.send(/* "You borrowed ..." */ null);
    }

    public void returnCopy(String loanId) { /* SQL + computeFine + mail */ }

    public BigDecimal computeFine(String loanId, LocalDate today) {
        Map<String, Object> loan = jdbc.queryForMap(
            "select * from loan where id = ?", loanId);
        long late = ChronoUnit.DAYS.between(
            (LocalDate) loan.get("due_on"), today);
        if ("STAFF".equals(loan.get("member_type"))) return BigDecimal.ZERO;
        BigDecimal fee = BigDecimal.valueOf(0.5).multiply(
            BigDecimal.valueOf(late));
        return fee.min(BigDecimal.valueOf(20));
    }

    public void chargeFine(String fineId, String cardToken) {
        // Stripe call, then "update fine set paid = true"
    }
    public void sendOverdueNotices() {
        // for each overdue loan: if pref == 'SMS' sms.send(...) else
```

```
    mail.send(...)
        }
        public void exportCatalogueCsv(Path target)   { /* SQL + file I/O */ }
        public void printMonthlyReport(YearMonth m)   { /* SQL + pdf */ }
        public void registerMember(MemberForm form)   { /* SQL + welcome mail */ }
    }
```

STEPS

1. Draw `diagrams/task5-before.mmd` exactly as the code is: `LibraryManager` with its five fields as associations to the concrete classes (`JdbcTemplate`, `JavaMailSender`, `TwilioClient`, `StripeClient`, `PdfWriter`) and its nine operations. Every arrow leaves `service` and lands on a technology class.
2. In the README, list at least six **smells** with the SOLID letter each violates and a one-line explanation. Examples to find: the `"STUDENT"`/`"STAFF"` string switch, the direct dependency on `StripeClient`, the mixing of SQL, mail and PDF in one class, the `if pref == 'SMS'` branch, the fine rule that lives in three places.
3. Design `diagrams/task5-after.mmd` reusing Tasks 3 and 4: `LoanService`, `FineService`, `NotificationService`, `CatalogueService`, `ReportService` in `service`; the interfaces `LoanRepository`, `MemberRepository`, `FeePolicy`, `NotificationChannel`, `PaymentGateway` in `domain`; `JpaLoanRepository`, `EmailChannel`, `SmsChannel`, `StripePaymentGateway`, `PdfReportWriter` as realizations. The loan limit and period become polymorphic operations of `Member`.
4. Check the after-diagram with the rule from slide 24: every arrow that leaves a `service` class ends in an `«interface»` or entity of `domain`. No `service` class has more than four associations; if it does, split it.
5. Fill in the **refactoring table** in the README (skeleton below), at least six rows, each with the principle, the before and the after element.
6. Implement one row for real: replace `computeFine` by `FineService.assess(Loan, LocalDate)` that delegates to the injected `FeePolicy`, and configure `DailyFeePolicy` as the default bean with `@Bean` in `config/LibraryConfig`. Write `FineServiceTest` with a fake `FeePolicy` (a lambda-like anonymous subclass) proving that the service adds no rule of its own.
7. Render both PNGs. Commit the god class unchanged under `legacy/` annotated `@Deprecated`, so reviewers can compare.

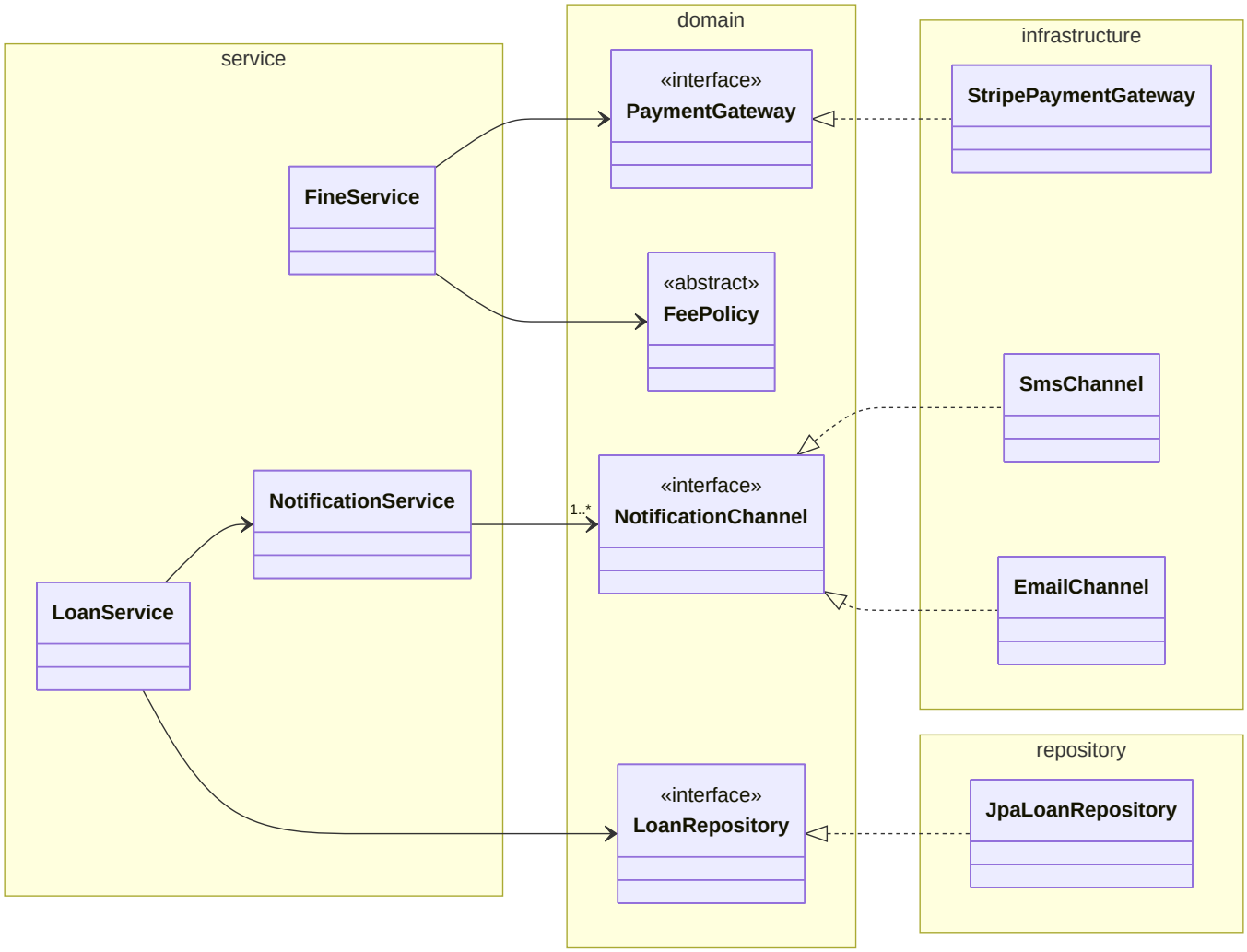
STARTER: REFACTORING TABLE

```
| # | Change                                                | Principle | Before
| After                                    |
|---|-------------------------------------------------------|-----------|---------
```

```
----------------------|----------------------------------------|
| 1 | Split LibraryManager into five services             | SRP       | 1 class,
9 operations, 5 techs | LoanService, FineService, ...           |
| 2 | Member type string switch -> polymorphic maxLoans() | OCP, LSP  |
"STUDENT".equals(type) ? 5 :.. | Member.maxLoans() overridden            |
| 3 |                                                     |           |
|                                       |
| 4 |                                                     |           |
|                                       |
| 5 |                                                     |           |
|                                       |
| 6 |                                                     |           |
|                                       |
```

EXPECTED OUTPUT

The core of the after-diagram (members hidden). Services on the left point only at domain interfaces; adapters on the right realize them.




```
$ mvn -q test -Dtest=FineServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
  assessDelegatesToPolicy          fake policy returned 7.00, service returned
```

```
  7.00
    onTimeLoanProducesNoFine        Optional.empty()
```

Show hints

- SRP smell test: describe the class in one sentence without using "and". "LibraryManager lends copies *and* computes fines *and* sends mail *and* charges cards *and* prints reports" fails.
- OCP: adding a third member type (*Alumni*, 2 loans, 7 days) must not touch `LoanService`. If you would edit a switch, it is not closed.
- DIP: `StripeClient` and `TwilioClient` are details. The domain defines `PaymentGateway` and `NotificationChannel`; `infrastructure` depends on `domain`, never the reverse.
- ISP: a `ReportService` that only reads loans should not receive an interface with `save()`. Consider `LoanQueries` separate from `LoanRepository`.
- LSP: `computeFine` returning zero for staff is a hidden special case. A `StaffFeePolicy` subclass whose `fee()` returns `Money.ZERO` keeps the contract; a subclass that throws would break it.

ACCEPTANCE CHECKLIST

- [ ] The before-diagram is faithful to the code (five technology associations, nine operations) and the README names 6+ smells with their SOLID letter.
- [ ] In the after-diagram every arrow leaving `service` ends in `domain`; the member-type switch is gone, replaced by polymorphism.
- [ ] The refactoring table has 6+ rows, each mapping a concrete change to a principle with before and after elements.
- [ ] `FineService` delegates to an injected `FeePolicy`; `FineServiceTest` passes with a fake policy and no Spring context.

★ Challenge: reverse-engineer, reconcile, decide Hard

90–120 min · optional, extra credit

SCENARIO

The faculty already runs a Spring Boot prototype of the library, written before any diagram existed. You receive a text inventory of its 15 classes (below). Reverse-engineer a design-level class diagram from it, compare it with your forward design from Tasks 3 to 5, produce a reconciled diagram with stereotypes, and record two design decisions as Architecture Decision Records (ADRs).

THE EXISTING CODEBASE (INVENTORY)

```
edu.itc.library.web
 1  MemberController        fields: MemberService memberService
                            methods: register(RegisterMemberRequest)
ResponseEntity<MemberDto>
                                     get(UUID id) MemberDto
 2  LoanController          fields: LoanService loanService
                            methods: borrow(BorrowRequest) LoanDto ;
giveBack(UUID loanId) LoanDto
 3  RegisterMemberRequest   record: String name, String email, String kind,
String studentNo,
                                    String department
 4  MemberDto               record: UUID id, String name, String kind, int
activeLoans

edu.itc.library.service
 5  MemberService           fields: MemberRepository members; JavaMailSender
mail
                            methods: register(RegisterMemberRequest) Member ;
find(UUID) Member
 6  LoanService             fields: LoanRepository loans; CopyRepository copies;
                                    MemberRepository members; Clock clock;
FeePolicy feePolicy;
                                    JavaMailSender mail
                            methods: borrow(UUID memberId, String barcode) Loan
                                     giveBack(UUID loanId) Loan
 7  FineService             fields: FineRepository fines; StripeClient stripe
                            methods: pay(UUID fineId, String cardToken) Fine

edu.itc.library.domain
 8  Member (abstract)       @Entity @Inheritance(JOINED)
                            fields: UUID id; String name; String email;
                                    @OneToMany(mappedBy="member") List<Loan>
loans
                            methods: maxLoans()* int ; loanPeriodDays()* int ;
                                     borrow(Copy, LocalDate) BorrowResult
 9  Student extends Member  fields: String studentNo   methods: maxLoans()=5,
loanPeriodDays()=14
10  Staff extends Member    fields: String department  methods: maxLoans()=10,
loanPeriodDays()=30
11  Loan                    @Entity  fields: UUID id; @ManyToOne Member member;
                                     @ManyToOne Copy copy; LocalDate borrowedOn,
dueOn, returnedOn
12  Copy                    @Entity  fields: String barcode; CopyStatus status;
                                     @ManyToOne Book book
13  Book                    @Entity  fields: String isbn; String title;
                                     @OneToMany(mappedBy="book", cascade=ALL,
                                                orphanRemoval=true) List<Copy>
```

```
copies

edu.itc.library.repository
14  JpaLoanRepository       interface extends JpaRepository<Loan, UUID>,
LoanRepository
                            (LoanRepository, MemberRepository, CopyRepository,
FineRepository
                             are interfaces in domain; the other Jpa* adapters
look the same)

edu.itc.library.infrastructure
15  EmailChannel            implements NotificationChannel ; fields:
JavaMailSender sender
                            (NotificationChannel is an interface in domain;
nothing injects it yet)
```

REQUIREMENTS

1. **Reverse-engineer** `diagrams/challenge-asis.mmd`: all 15 classes plus the four domain interfaces, in namespaces, with every field that is a class turned into an association (with multiplicity) and every parameter type into a dependency. Apply the stereotypes `«controller»`, `«service»`, `«entity»`, `«repository»`, `«dto»` as Mermaid annotations.
2. **Reconcile**: write a discrepancy table (README) with at least six rows comparing the as-is diagram with your Task 4/5 design, for example: `Member.email` is a `String` instead of the `Email` record; `Book/Copy` is bidirectional (`mappedBy`) where your design was one-way; `LoanService` has six dependencies including `JavaMailSender`; `FineService` depends on `StripeClient`; `RegisterMemberRequest.kind` is a string discriminator; `Fine` and `Reservation` are missing; `EmailChannel` exists but nothing uses it.
3. **Design-level target** `diagrams/challenge-design.mmd`: the reconciled diagram with stereotypes, keeping what is good in the prototype (the `Member` hierarchy, `BorrowResult`) and fixing the discrepancies you decided to fix. Each fix must trace to a row of the table.
4. **Two ADRs** in `README.md` (or `docs/adr/0001-….md`, `0002-….md`) using the skeleton below. Suggested subjects: *"Book-Copy association is unidirectional"* versus the prototype's bidirectional mapping, and *"Notifications go through NotificationChannel, services never see JavaMailSender"*. Each ADR names at least one alternative and one consequence you accept.
5. Implement the two ADRs in `library/` so that the code matches `challenge-design.mmd`; `mvn -q verify` passes.

SKELETON

```
classDiagram
  direction LR
```

```
    namespace web {
      class LoanController {
        <<controller>>
        +borrow(req: BorrowRequest) LoanDto
        +giveBack(loanId: UUID) LoanDto
      }
    }
    namespace service {
      class LoanService {
        <<service>>
        +borrow(memberId: UUID, barcode: String) Loan
      }
    }
    namespace domain {
      class Loan {
        <<entity>>
      }
      class LoanRepository {
        <<repository>>
      }
    }
    LoanController --> LoanService
    LoanService --> LoanRepository
    LoanService ..> Loan : «create»

  # ADR-0001: <title in the form of a decision>
  Status: accepted | Date: 2026-09-25
  ## Context      what forces are at play (use cases, JPA, team habits)
  ## Decision     one sentence, active voice ("We map Book-Copy one-way ...")
  ## Alternatives what else was considered and why it lost
  ## Consequences what becomes easier, what becomes harder, what we must watch
```

EXPECTED OUTPUT

```
  diagrams/challenge-asis.mmd      19 classifiers, 5 namespaces, stereotypes on
  every class
  diagrams/challenge-design.mmd    same plus Fine, Reservation, PaymentGateway,
  NotificationChannel
                                   wired; LoanService has <= 4 associations, all
  into domain
  README.md                        discrepancy table (6+ rows), ADR-0001, ADR-0002
  $ mvn -q verify
  [INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
  [INFO] BUILD SUCCESS
```

Show hints

- Reverse-engineering rule: a field whose type is one of the 19 classifiers is an association; a field of a framework type (`Clock`, `JavaMailSender`) is an association to an external class, which is exactly the DIP smell you want to expose in the as-is diagram.
- The stereotypes are annotations in Mermaid (`<<service>>` on the first body line). They are the same words Spring uses (`@Controller`, `@Service`, `@Repository`, `@Entity`), which is no coincidence.
- Bidirectional `Book/Copy` is not wrong; it costs a second field to keep in sync. Your ADR should say what the navigation is used for (UC-02 counts available copies per book, UC-03 only needs the copy). Decide, do not hedge.
- An ADR is short: half a page. The value is in *Alternatives* and *Consequences*; a decision without a rejected alternative is a description, not a decision.
- Keep `challenge-asis.mmd` ugly on purpose. A faithful picture of the problem is what convinces reviewers to accept the design.

ACCEPTANCE CHECKLIST

- [ ] The as-is diagram contains all 19 classifiers with stereotypes, every class-typed field as an association and every parameter type as a dependency.
- [ ] The discrepancy table has 6+ rows and every fix in the design diagram traces to one of them.
- [ ] Two ADRs with Context, Decision, Alternatives and Consequences; the code implements both decisions.
- [ ] `mvn -q verify` passes and the design diagram obeys the rule "every arrow leaving `service` ends in `domain`".

Grading rubric

Component	Points	Full marks when…
Task 1 · Domain model	10	8 to 12 classes traceable to the noun table; typed attributes; value objects and enumerations annotated; no accessors.
Task 2 · Associations	15	Every line has both multiplicities and a role; navigability follows the use cases; the justification table supports the diamonds chosen.
Task 3 · Generalization and interfaces	20	Abstract `Member` with polymorphic limits, interfaces with realizations, `FeePolicy` template method; Java matches the diagram; `FeePolicyTest` green.

Component	Points	Full marks when…
Task 4 · Packages and implementation	20	Five namespaces with inward dependencies; JPA mapping translates composition, inheritance and value objects exactly; sealed `BorrowResult`; `MemberLoanLimitTest` green without Spring.
Task 5 · SOLID review	25	Faithful before-diagram, 6+ smells with principles, after-diagram with all service arrows into `domain`, complete refactoring table, `FineService` + test implemented.
Code / diagram quality	10	Diagram sources render without warnings; consistent naming between diagrams and code; README readable; commits per task.
★ Challenge (extra credit)	+20	As-is diagram complete with stereotypes, 6+ reconciled discrepancies, two real ADRs with alternatives and consequences, code matches the design and `mvn verify` passes.
Total	**100 (+20)**	

Automatic deductions: −3 per association end without multiplicity (max −15); −5 per generalization or realization triangle drawn at the wrong end; −5 if any class lists getters/setters as operations; −10 if the Java code contradicts the diagram (composition drawn but the list is exposed mutable or not cascaded; `«interface»` drawn but the JPA class is injected); −10 if a diagram is submitted as PNG only, without its `.mmd`/`.puml` source; −10 for committed `target/` or IDE folders; −5 if `service` imports `repository` or `infrastructure`.

Submission

1. Repository layout:

```
lab-08/
├── README.md                          noun table · justification table · Task
3 paragraph
│                                      translation table · smells · refactoring
table · ADRs
├── diagrams/
│   ├── task1-domain.mmd               + .png
│   ├── task2-associations.mmd         + .png
│   ├── task3-hierarchy.mmd            + .png
```

```
│   ├── task4-packages.mmd          + .png
│   ├── task5-before.mmd            + .png
│   ├── task5-after.mmd             + .png
│   ├── challenge-asis.mmd          + .png   (challenge)
│   └── challenge-design.mmd        + .png   (challenge)
└── library/
    ├── pom.xml
    └── src/main/java/edu/itc/library/{web,service,domain,repository,infrastructure,config,legacy}

    src/test/java/edu/itc/library/domain/{MemberLoanLimitTest,FeePolicyTest}.java
        src/test/java/edu/itc/library/service/FineServiceTest.java
```

2. Verification that must pass from a clean checkout:

```
cd lab-08
for f in diagrams/*.mmd; do
  npx -y @mermaid-js/mermaid-cli -i "$f" -o "${f%.mmd}.png" -b transparent -s 2
done
cd library && mvn -q verify
```

3. Branch `lab-08`, merge request titled `Lab-08 – <your name>`. In the MR description paste the refactoring table from Task 5 and link the two ADRs if you did the challenge.

Back to Chapter 08 slides | Retake the chapter quiz | ← Lab-07